

TP3 — Optimizacion de consultas: evidencia de codigo y planes reales

Captura por seccion con Win+Shift+S · Scripts de las partes 1, 2, 4 y 5 con EXPLAIN ANALYZE real antes/despues

TP3 · Parte 1 · carga_masiva.sql — desactivado de triggers (seed) y categorías base

Protocolo: BEGIN, DISABLE TRIGGER durante el seed, categorías ON CONFLICT DO NOTHING

```
-- =====
-- FOOD STORE - TP3 - Parte 1: carga masiva (seed voluminoso)
-- =====
-- Objetivo: poblar la base para que los planes de ejecución sean
-- lentos a escala real (Seq Scan) y la optimización sea medible.
--
-- Volumen objetivo (consigna del TP3):
-- * >= 50.000 productos distribuidos entre categorías existentes
-- * 20.000 clientes
-- * 200.000 pedidos con sus detalles (detalle_pedido)
--
-- PROTOCOLO DE SEGURIDAD (catedra): copia, transacción, respaldo.
-- Este script corre sobre food_store_tp3 (copia de trabajo),
-- NUNCA sobre plantilla_base. Previamente se tomó un pg_dump.
--
-- DECISION JUSTIFICADA sobre los triggers de la Parte 1 del TP2:
-- Las 3 restricciones (baja lógica, stock, subtotal) están
-- pensadas para operación transaccional de a uno. Durante un
-- bulk insert masivo el trigger de stock haría centenares de
-- miles de SELECT + UPDATE de una fila (lentísimo) y dejaría
-- los stocks en negativo. Lo profesional es DESACTIVAR los
-- triggers durante la carga y REACTIVARLOS al final, porque:
-- 1) La carga es un seed, no una venta real (no debe disparar
--    la lógica de negocio del TP2);
-- 2) se restaura el estado exacto antes/después.
-- Se documenta en la DUIA como decisión aceptada y reversible.
-- (No se desactiva trg_*_no_delete porque no se hace DELETE.)
-- =====
```

```
BEGIN;
```

```
-- -----
-- Paso 0: desactivar triggers de negocio durante el seed
-- -----
```

```
ALTER TABLE detalle_pedido DISABLE TRIGGER ALL;
ALTER TABLE producto       DISABLE TRIGGER ALL;
ALTER TABLE categoria       DISABLE TRIGGER ALL;
```

```
-- -----
-- Paso 1: categorías base (si no existen)
-- -----
```

```
INSERT INTO categoria (nombre) VALUES
    ('Pizzas'),
    ('Empanadas'),
    ('Bebidas'),
    ('Postres')
```

```
ON CONFLICT (nombre) DO NOTHING;
```

TP3 · Parte 1 · carga_masiva.sql — 50.000 productos + 20.000 clientes

generate_series, precios 500-5000, stock 0-200, repartidos en 4 categorías; 20.000 clientes

50.000 productos distribuidos entre categorías

```
-- Precio entre 500 y 5000, stock aleatorio 0..200.
```

```
-- generate_series de 1..50000; se reparten 4 categorías.
```

```
INSERT INTO producto (nombre, descripcion, precio, stock, activo, categoria_id)
```

```
SELECT
```

```
    'Producto ' || g,                                -- nombre unico
```

```
    'Descripcion generada para el producto ' || g,
```

```
    round((500 + random() * 4500)::numeric, 2),        -- precio 500..5000
```

```
    floor(random() * 201)::int,                        -- stock 0..200
```

```
    TRUE,
```

```
    (SELECT id FROM categoria WHERE nombre = (ARRAY['Pizzas','Empanadas','Bebidas','Postres'])[1 +
```

```
FROM generate_series(1, 50000) AS g;
```

```
-- Paso 3: 20.000 clientes
```

```
INSERT INTO cliente (nombre, apellido, email, telefono)
```

```
SELECT
```

```
    'Nombre' || g,
```

```
    'Apellido' || g,
```

```
    'cliente' || g || '@foodstore.com',
```

```
    '11' || lpad(g::text, 8, '0')
```

```
FROM generate_series(1, 20000) AS g;
```

TP3 · Parte 1 · carga_masiva.sql — 200.000 pedidos + 600.000 detalles y ANALYZE final

3 líneas por pedido con producto distinto (unicidad) + ANALYZE para estadísticas

```

-- Paso 4: 200.000 pedidos con detalles
-- Cada pedido tiene 1..4 líneas de detalle con cantidad 1..9.
-- El trigger de subtotal esta desactivado, pero igual cargamos
-- subtotal = cantidad * precio_unitario (consistente).
-- -----
WITH pedidos AS (
    SELECT
        g AS n,
        now() - (random() * interval '400 days') AS fecha,
        (ARRAY['EFECTIVO','TARJETA','TRANSFERENCIA'])[1 + floor(random()*3)::int)::forma_pago_enum
        1 + floor(random() * 20000)::int AS cliente_id
    FROM generate_series(1, 200000) AS g
)
INSERT INTO pedido (fecha, forma_pago, cliente_id)
SELECT fecha, fp, cliente_id FROM pedidos;

-- Detalles: para cada pedido, 2..4 líneas de detalle (cada una con
-- un producto distinto dentro del mismo pedido, respetando
-- uq_detalle_pedido_producto). cantidad 1..9, subtotal consistente.
-- Se usa generate_series para las líneas y se asigna un producto
-- diferente a cada línea del mismo pedido (desplazamiento por línea)
-- para evitar colisiones de unicidad dentro de un pedido.
-- Volumen: 200.000 pedidos x 3 líneas en promedio = ~600.000 detalles.
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
SELECT
    p.id,
    1 + ((p.id * 7 + linea) % 50000)::int AS producto_id, -- producto distinto por línea
    1 + floor(random() * 9)::int AS cantidad,
    pr.precio,
    pr.precio * (1 + floor(random() * 9)::int) AS subtotal
FROM generate_series(1, 200000) AS g
JOIN LATERAL (SELECT id FROM pedido WHERE id = g) p ON true
CROSS JOIN generate_series(0, 2) AS linea -- 3 líneas por pedido
CROSS JOIN LATERAL (
    SELECT precio FROM producto WHERE id = 1 + ((g * 7 + linea) % 50000)::int
) pr;

-- -----
-- Paso 5: ANALYZE para refrescar estadísticas del optimizador
-- -----
ANALYZE categoria;
ANALYZE producto;
ANALYZE cliente;
ANALYZE pedido;
ANALYZE detalle_pedido;

```

```
-- -----  
-- Paso 6: reactivar triggers de negocio (estado original)  
-- -----  
  
ALTER TABLE detalle_pedido ENABLE TRIGGER ALL;  
ALTER TABLE producto      ENABLE TRIGGER ALL;  
ALTER TABLE categoria      ENABLE TRIGGER ALL;  
  
COMMIT;  
  
-- -----  
-- Verificación de conteos (después del COMMIT)  
-- -----  
  
SELECT 'categoria' AS tabla, count(*) FROM categoria  
UNION ALL SELECT 'producto', count(*) FROM producto  
UNION ALL SELECT 'cliente',  count(*) FROM cliente  
UNION ALL SELECT 'pedido',   count(*) FROM pedido  
UNION ALL SELECT 'detalle_pedido', count(*) FROM detalle_pedido;
```

TP3 · Parte 1 · Verificación de conteos (salida real de psql)

Cumple y supera los mínimos de la consigna (50k/20k/200k)

tabla	count
-----+-----	
categoria	4
cliente	20000
producto	50000
pedido	200000
detalle_pedido	600000
(5 filas)	

TP3 · Parte 2 · consultas_lentas.sql — las 3 consultas de reporting

Sin índice útil en el esquema base: Seq Scan y agregaciones pesadas esperadas

```
-- =====
-- FOOD STORE - TP3 - Parte 2: consultas lentas y EXPLAIN ANALYZE
-- =====
-- Consultas elegidas (variantes propias sobre el modelo) que a
-- esta escala vuelven lentas. Antes de cualquier cambio se guarda
-- el plan completo de cada una.
--
-- Los tres son queries de REPORTING reales sobre el modelo, de las
-- que el optimizador no puede resolver con los índices del esquema
-- base (schema.sql) y por eso producen Seq Scan / agregaciones
-- pesadas sobre las 600.000 líneas de detalle_pedido.
-- =====

-- CONSULTA 1: resumen de ventas por producto (unidades y total).
-- Escanea 600.000 líneas de detalle, JOIN con producto y agrega por
-- producto. No hay índice útil para la agregación; usa HashAggregate
-- con spool a disco.
EXPLAIN ANALYZE
SELECT pr.nombre,
       count(*)      AS líneas,
       sum(dp.cantidad) AS unidades,
       sum(dp.subtotal) AS total_vendido
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
GROUP BY pr.nombre
ORDER BY total_vendido DESC
LIMIT 20;

-- CONSULTA 2: facturación por rango de fechas (reporte mensual).
-- Filtra pedidos por fecha y los cruza con su cliente para el
-- listado de facturas. No existe índice sobre pedido.fecha, por lo
-- que se hace Seq Scan sobre las 200.000 filas de pedido.
EXPLAIN ANALYZE
SELECT p.id AS nro_factura, p.fecha, p.forma_pago, c.nombre AS cliente
FROM pedido p
JOIN cliente c ON c.id = p.cliente_id
WHERE p.fecha >= '2025-01-01'
      AND p.fecha < '2026-01-01'
ORDER BY p.fecha, p.id;

-- CONSULTA 3: total vendido por mes (historicos).
-- Cruza detalle (600.000 filas) con pedido para filtrar por fecha y
-- luego agrega por mes (date_trunc). Agregación pesada de reporte.
EXPLAIN ANALYZE
SELECT to_char(date_trunc('month', p.fecha), 'YYYY-MM') AS mes,
       sum(dp.subtotal)                                AS total_mes,
```

```

count(distinct p.id) AS pedidos_mes
FROM detalle_pedido dp
JOIN pedido p ON p.id = dp.pedido_id
GROUP BY date_trunc('month', p.fecha)
ORDER BY mes;

```

TP3 · Parte 2 · PLAN ANTES — Consulta 1 (resumen de ventas): Parallel Seq Scan + spool a disco — 994 ms

Nodo dominante: Parallel Seq Scan (600.000 filas) y HashAggregate con spool a disco

```

Limit (cost=60128.04..60128.09 rows=20 width=62) (actual time=964.453..999.509 rows=20.00 loops=1)
  -> Sort ... Sort Method: top-N heapsort Memory: 27kB
    -> Finalize GroupAggregate (cost=42696.58..58797.56 rows=50000)
      Group Key: pr.nombre
      -> Gather Merge (cost=42696.58..56672.56 rows=120000)
        -> Partial HashAggregate ... Batches: 5 Disk Usage: 8241kB
          -> Hash Join
            -> Parallel Seq Scan on detalle_pedido dp
            -> Hash ... -> Seq Scan on producto pr

Planning Time: 8.544 ms
Execution Time: 993.806 ms

```

TP3 · Parte 2 · PLAN ANTES — Consulta 2 (facturacion por fecha): Seq Scan pedido — 114 ms

Recorre 200.000 pedidos para quedarse con 78.840 del rango (no existia indice por fecha)

```

Sort (cost=13805.87..14001.15 rows=78111 width=31) (actual time=88.252..94.711 rows=77840.00 loops=1)
  Sort Method: external merge Disk: 3168kB
  -> Hash Join
    Hash Cond: (p.cliente_id = c.id)
    -> Seq Scan on pedido p
      Filter: ((fecha >= ...) AND (fecha < ...))
      Rows Removed by Filter: 122160
    -> Hash -> Seq Scan on cliente c

Planning Time: 8.640 ms
Execution Time: 113.874 ms

```

TP3 · Parte 2 · PLAN ANTES — Consulta 3 (total por mes): sort a disco 27 MB — 1.734 ms

Lo más caro: ordenar 600.000 filas por mes con sort a disco (27 MB)

```
Sort (cost=137102.48..137602.48 rows=200000 width=80) (actual time=1711.741..1711.746 rows=15.00)
  Sort Method: quicksort
    -> GroupAggregate
      Group Key: (date_trunc('month'::text, p.fecha))
      -> Sort (cost=100602.84..102102.84 rows=600000 width=32)
        Sort Method: external merge Disk: 27136kB
          -> Hash Join (dp.pedido_id = p.id)
            -> Seq Scan on detalle_pedido dp rows=600000
Planning Time: 1.332 ms
Execution Time: 1734.203 ms
```

TP3 · Parte 2 · optimizaciones.sql — índice de cobertura + índice por fecha

idx_detalle_producto_cover (producto_id) INCLUDE (cantidad, subtotal) + idx_pedido_fecha

```

-- =====
-- FOOD STORE - TP3 - Parte 2: cambios propuestos por la IA
-- =====
-- Cada cambio justificado en terminos del plan REAL obtenido con
-- consultas_lentas.sql. Se aplica sobre la copia de trabajo y se
-- vuelve a medir con EXPLAIN ANALYZE.
--
-- CONSULTA 1 - resumen de ventas por producto:
-- El plan hace "Parallel Seq Scan on detalle_pedido dp"
-- (600.000 filas) + HashAggregate con spool a disco
-- (Batches: 5, Disk Usage ~7MB). El Query Scan de detalle lee
-- toda la tabla y despues agrega.
-- PROPUUESTA: indice de cobertura que responda la agregacion con
-- Index Only Scan, evitando leer las filas del heap.
-- CREATE INDEX idx_detalle_producto_cover
--     ON detalle_pedido (producto_id) INCLUDE (cantidad, subtotal);
-- Ataca el nodo "Seq Scan on detalle_pedido".
--
-- CONSULTA 2 - facturacion por rango de fechas:
-- El plan hace "Seq Scan on pedido p" con
-- "Filter: (fecha >= ... AND fecha < ...)"
-- y "Rows Removed by Filter: 122160" (escanea 200.000 filas
-- para quedarse con 78.840). Actualmente NO hay indice por fecha.
-- PROPUUESTA: CREATE INDEX ON pedido (fecha);
-- Ataca el nodo "Seq Scan on pedido" -> Bitmap Index Scan.
--
-- CONSULTA 3 - total vendido por mes:
-- Lo mas caro es el "Sort Method: external merge Disk: 27136kB"
-- sobre las 600.000 filas y el "Hash Join" con
-- "Seq Scan on detalle_pedido dp". La Group Key es
-- date_trunc('month', fecha): si el pedido viene ordenado por
-- fecha (indice), el GroupAggregate puede leer en orden y parte
-- del sort desaparece.
-- PROPUUESTA: se agrega (INDEX en pedido(fecha)) que tambien
-- alimenta a la consulta 3 ordenando por fecha.
-- =====

BEGIN;

CREATE INDEX idx_detalle_producto_cover
    ON detalle_pedido (producto_id) INCLUDE (cantidad, subtotal);

CREATE INDEX idx_pedido_fecha
    ON pedido (fecha);

ANALYZE detalle_pedido;

```

```
ANALYZE pedido;  
  
COMMIT;  
  
SELECT 'indices creados' AS estado;
```

TP3 · Parte 2 · Reescritura aplicada a Consulta 1 — agrupar por producto_id con Index Only Scan

El GROUP BY por producto_id (columna indexada) permite Index Only Scan; el join con producto se hace al final

```
SELECT pr.nombre, v.unidades, v.total_vendido  
FROM (  
    SELECT producto_id, sum(cantidad) AS unidades, sum(subtotal) AS total_vendido  
    FROM detalle_pedido  
    GROUP BY producto_id  
) v  
JOIN producto pr ON pr.id = v.producto_id  
ORDER BY v.total_vendido DESC  
LIMIT 20;
```

TP3 · Parte 2 · PLAN DESPUES — Consulta 1: Index Only Scan (Heap Fetches: 0) — 290 ms (3,4x)

Cost 60128→29894 · tiempo 994→290 ms · mejora 3,4x

```
Limit (cost=29894.85..29894.90 rows=20 width=54) (actual time=289.173..289.178 rows=20.00 loops=1)  
  -> Sort ... Sort Method: top-N heapsort  
    -> Hash Join  
      -> GroupAggregate  
        Group Key: detalle_pedido.producto_id  
        -> Index Only Scan using idx_detalle_producto_cover on detalle_pedido  
          Heap Fetches: 0  
          Index Searches: 1  
      -> Hash -> Seq Scan on producto pr  
Planning Time: 8.230 ms  
Execution Time: 289.977 ms
```

TP3 · Parte 2 · PLAN DESPUES — Consulta 2: Bitmap Index Scan on idx_pedido_fecha — 124 ms

Plan cambiado (Seq→Bitmap) pero tiempo similar: rango del 40,9% de la tabla poco selectivo (documentado)

```
Sort (cost=13656.33..13851.83 rows=78199 width=31) (actual time=98.845..105.558 rows=77840.00 loops=1)
  Sort Method: external merge  Disk: 3168kB
  -> Hash Join
    -> Bitmap Heap Scan on pedido p
      Recheck Cond: ((fecha >= ...) AND (fecha < ...))
    -> Bitmap Index Scan on idx_pedido_fecha
      Index Cond: ((fecha >= ...) AND (fecha < ...))
Planning Time: 13.504 ms
Execution Time: 124.416 ms
```

TP3 · Parte 2 · PLAN DESPUES — Consulta 3: sin mejora, se descarta con registro (1552 ms)

El índice por fecha no alimenta el Group Key (join por pedido_id, no por fecha). Se documenta, no se descarta en silencio

```
Sort (cost=137102.48..137602.48 rows=200000) (actual time=1526.215..1526.219 rows=15.00 loops=1)
  -> GroupAggregate
    Group Key: (date_trunc('month'::text, p.fecha))
    -> Sort (cost=100602.84..102102.84 rows=600000 width=32)
      Sort Method: external merge  Disk: 27136kB
      -> Hash Join (dp.pedido_id = p.id)
Planning Time: 0.518 ms
Execution Time: 1551.634 ms
```

TP3 · Parte 4 · Spec 1 — agregacion: LEFT JOIN (AI) vs subconsulta escalar (alternativa) + EXCEPT

Equivalencia verificada: (A)EXCEPT(B)=0 y (B)EXCEPT(A)=0

```
-- =====
-- FOOD STORE - TP3 - Parte 4: consultas bajo spec + equivalencia
-- =====
-- Se redacta una spec precisa, la IA genera el SQL a partir de la
-- spec (sin solución previa), y se escribe una SEGUNDA versión de
-- distinta estructura que responda la misma pregunta. Luego se
-- verifica equivalencia con EXCEPT en ambas direcciones (debe dar
-- 0 filas). Sobre food_store_tp3.
-- =====

-- =====
-- SPEC 1 (agregación / resumen)
-- «Genera una consulta SQL sobre el esquema Food Store que devuelva,
-- para cada categoría VIGENTE (categoría.activo = FALSE son las
-- dadas de baja y NO deben aparecer), el nombre de la categoría y
-- la cantidad de productos VIGENTES (producto.activo = TRUE) que
-- tiene, incluyendo las categorías sin productos vigentes con
-- cantidad 0. Ordena de mayor a menor cantidad (y por nombre como
-- desempate asc). No uses SELECT *. Usa LEFT JOIN.»
-- =====

-- VERSION A (generada por la IA a partir de la spec)
SELECT c.nombre, count(p.id) AS cantidad_productos
FROM categoria c
LEFT JOIN producto p ON p.categoria_id = c.id AND p.activo = TRUE
WHERE c.activo = TRUE
GROUP BY c.id, c.nombre
ORDER BY cantidad_productos DESC, c.nombre ASC;

-- VERSION B (alternativa de distinta estructura: subconsulta en vez
-- de LEFT JOIN con condición en el ON)
SELECT c.nombre,
       (SELECT count(*) FROM producto p
        WHERE p.categoria_id = c.id AND p.activo = TRUE) AS cantidad_productos
FROM categoria c
WHERE c.activo = TRUE
ORDER BY cantidad_productos DESC, c.nombre ASC;

-- VERIFICACION DE EQUIVALENCIA SPEC 1 (ambas deben dar 0 filas)
(
  SELECT c.nombre, count(p.id) AS cantidad_productos
  FROM categoria c
  LEFT JOIN producto p ON p.categoria_id = c.id AND p.activo = TRUE
  WHERE c.activo = TRUE
  GROUP BY c.id, c.nombre
```

```
) EXCEPT (  
  SELECT c.nombre,  
         (SELECT count(*) FROM producto p  
          WHERE p.categoria_id = c.id AND p.activo = TRUE) AS cantidad_productos  
  FROM categoria c  
  WHERE c.activo = TRUE  
);  
  
(  
  SELECT c.nombre,  
         (SELECT count(*) FROM producto p  
          WHERE p.categoria_id = c.id AND p.activo = TRUE) AS cantidad_productos  
  FROM categoria c  
  WHERE c.activo = TRUE  
)  
EXCEPT (  
  SELECT c.nombre, count(p.id) AS cantidad_productos  
  FROM categoria c  
  LEFT JOIN producto p ON p.categoria_id = c.id AND p.activo = TRUE  
  WHERE c.activo = TRUE  
  GROUP BY c.id, c.nombre  
);  
  
-- =====
```

TP3 · Parte 4 · Spec 2 — subconsulta: HAVING-subconsulta (AI) vs CTE+join (alternativa) + EXCEPT

Equivalencia verificada: ambos EXCEPT = 0 filas

```
-- SPEC 2 (subconsulta)
-- «Genera una consulta SQL sobre el esquema Food Store que liste los
-- productos VIGENTES (producto.activo = TRUE) cuyo total vendido
-- (suma de subtotal en detalle_pedido) sea MAYOR que el promedio
-- del total vendido de TODOS los productos vigentes. Devuelve el
-- nombre del producto y su total vendido con dos decimales,
-- ordenado de mayor a menor total. No uses SELECT *.»
-- =====

-- VERSION A (generada por la IA a partir de la spec)
SELECT pr.nombre, round(sum(dp.subtotal), 2) AS total_vendido
FROM producto pr
JOIN detalle_pedido dp ON dp.producto_id = pr.id
WHERE pr.activo = TRUE
GROUP BY pr.id, pr.nombre
HAVING sum(dp.subtotal) > (
    SELECT avg(vendido)
    FROM (
        SELECT sum(dp2.subtotal) AS vendido
        FROM producto pr2
        JOIN detalle_pedido dp2 ON dp2.producto_id = pr2.id
        WHERE pr2.activo = TRUE
        GROUP BY pr2.id
    ) t
)
ORDER BY total_vendido DESC;

-- VERSION B (alternativa de distinta estructura: CTE + join en vez
-- de subconsulta en el HAVING)
WITH ventas AS (
    SELECT dp.producto_id, sum(dp.subtotal) AS total_vendido
    FROM detalle_pedido dp
    GROUP BY dp.producto_id
),
promedios AS (
    SELECT avg(total_vendido) AS promedio
    FROM ventas v
    JOIN producto pr ON pr.id = v.producto_id
    WHERE pr.activo = TRUE
)
SELECT pr.nombre, round(v.total_vendido, 2) AS total_vendido
FROM ventas v
JOIN producto pr ON pr.id = v.producto_id
CROSS JOIN promedios
WHERE pr.activo = TRUE
```

```

    AND v.total_vendido > promedios.promedio
ORDER BY v.total_vendido DESC;

-- VERIFICACION DE EQUIVALENCIA SPEC 2 (ambas deben dar 0 filas)
(
    SELECT pr.nombre, round(sum(dp.subtotal), 2) AS total_vendido
    FROM producto pr
    JOIN detalle_pedido dp ON dp.producto_id = pr.id
    WHERE pr.activo = TRUE
    GROUP BY pr.id, pr.nombre
    HAVING sum(dp.subtotal) > (
        SELECT avg(vendido)
        FROM (
            SELECT sum(dp2.subtotal) AS vendido
            FROM producto pr2
            JOIN detalle_pedido dp2 ON dp2.producto_id = pr2.id
            WHERE pr2.activo = TRUE
            GROUP BY pr2.id
        ) t
    )
)
) EXCEPT (
    WITH ventas AS (
        SELECT dp.producto_id, sum(dp.subtotal) AS total_vendido
        FROM detalle_pedido dp
        GROUP BY dp.producto_id
    ),
    promedios AS (
        SELECT avg(total_vendido) AS promedio
        FROM ventas v
        JOIN producto pr ON pr.id = v.producto_id
        WHERE pr.activo = TRUE
    )
    SELECT pr.nombre, round(v.total_vendido, 2) AS total_vendido
    FROM ventas v
    JOIN producto pr ON pr.id = v.producto_id
    CROSS JOIN promedios
    WHERE pr.activo = TRUE
        AND v.total_vendido > promedios.promedio
);

(
    WITH ventas AS (
        SELECT dp.producto_id, sum(dp.subtotal) AS total_vendido
        FROM detalle_pedido dp
        GROUP BY dp.producto_id
    ),
    promedios AS (
        SELECT avg(total_vendido) AS promedio
        FROM ventas v
        JOIN producto pr ON pr.id = v.producto_id

```

```
WHERE pr.activo = TRUE
)
SELECT pr.nombre, round(v.total_vendido, 2) AS total_vendido
FROM ventas v
JOIN producto pr ON pr.id = v.producto_id
CROSS JOIN promedios
WHERE pr.activo = TRUE
AND v.total_vendido > promedios.promedio
) EXCEPT (
SELECT pr.nombre, round(sum(dp.subtotal), 2) AS total_vendido
FROM producto pr
JOIN detalle_pedido dp ON dp.producto_id = pr.id
WHERE pr.activo = TRUE
GROUP BY pr.id, pr.nombre
HAVING sum(dp.subtotal) > (
    SELECT avg(vendido)
    FROM (
        SELECT sum(dp2.subtotal) AS vendido
        FROM producto pr2
        JOIN detalle_pedido dp2 ON dp2.producto_id = pr2.id
        WHERE pr2.activo = TRUE
        GROUP BY pr2.id
    ) t
)
);
```

TP3 · Parte 5 · Competencia: índice compuesto (categoria_id, precio) — 39,9 ms → 5,7 ms (7,0×)

Con ambos filtros convertidos en Index Cond desaparece el descarte de 6.904 filas · mejora 7,0×

```
SELECT c.nombre AS categoria, p.nombre, p.precio, p.stock
FROM producto p
JOIN categoria c ON c.id = p.categoria_id
WHERE c.nombre = 'Pizzas'
      AND p.precio BETWEEN 1000 AND 3000
ORDER BY p.precio DESC;
```

-- Solucion aplicada

```
CREATE INDEX idx_producto_cat_precio ON producto (categoria_id, precio);
```

PLAN ANTES: Bitmap Index S
Filter: (precio >= 1000 AND precio <= 3000)

Rows Removed by Filter: 6904

Execution Time: 39.897 ms

PLAN DESPUES: Bitmap Index Scan on idx_producto_cat_precio

Index Cond: categoria_id=c.id AND precio>=1000 AND precio<=3000

Execution Time: 5.737 ms